

Práctica 4

Primer caso de uso - Login

Diana Laura Luciano Paniagua
dianaluciano@ciencias.unam.mx

Luis Fernando Yang Fong Baeza
fernandofong@ciencias.unam.mx

Valeria García Landa
vale.garcia.landa@gmail.com

Francisco Valdés Souto
fvaldes@ciencias.unam.mx

3 de abril de 2026

1. Introducción

Un *login* es la base para cualquier sistema web puesto que su funcionalidad puede depender de la autenticación de un usuario, ya que esto relaciona directamente con un *actor* de cualquier caso de uso. El propósito de esta práctica es que el alumno haga la base (un login) para su sistema web que entregará al final del curso.

Este *login* tiene que ser utilizando las herramientas que se vieron durante el laboratorio, o sea, Python en versión $\geq 3,12.x$, MySQL $\geq 9,5.x$, Flask SQL Alchemy y Flask.

Nota: Aunque el sistema web se entregue de manera grupal, cada alumno debe de entregar su propia práctica para aprender los conocimientos necesarios para implementar un caso de uso tan importante y que abarca tanto como lo es un *login*. Posteriormente cada equipo deberá decidir cuál es la mejor base para empezar su proyecto y construir sobre él.

1.1. Entregados

Los entregados para esta práctica son:

1. PDF de la práctica
2. Plantilla de L^AT_EX práctica para responder las preguntas.
3. Archivo de .sql

2. Desarrollo

2.1. Preparación de la práctica

En esta ocasión, no se le entrega un código fuente al alumno puesto que es su responsabilidad levantar un proyecto desde cero, al igual que es su deber levantar la base de datos que les fue proporcionada por los ayudantes de laboratorio, esto para un mejor aprendizaje.

2.2. Realización de la práctica

Para realizar la práctica, el alumno deberá de crear una aplicación en Flask que muestre una pantalla de inicio, que capture al menos dos datos, usuario y contraseña y que dicha pantalla permita enviar la información capturada con un botón. En caso de que el usuario y contraseña sean correctos, la aplicación debe de presentar una siguiente pantalla que debe de mostrar el nombre del usuario y el rol si es que la información ingresada fue correcta, como mínimo. La validez de la información tiene que ser con respecto a lo que está definido en el

script de SQL proporcionado por los ayudantes de laboratorio. El estilo de las pantallas (CSS) es a completo criterio del alumno siempre y cuando no se caiga en obscenidades.

En el script de la base de datos que se les fue proporcionada, se incluyen dos usuarios pero con roles diferentes, un usuario y un admin, el sistema tiene que diseñar una manera de diferenciar ambos roles, ya sea con vistas diferentes o inyectando diferentes valores pero tiene que ser una diferencia visible. En caso de que haya un error en usuario, contraseña o ambos, el sistema debe de también lograr representarlo en una vista pero permitiendo al usuario volver al inicio del flujo.

Por último, también es obligación del alumno solicitar a alguna herramienta de IA que le cree un HTML y CSS para sus vistas o *templates* de su proyecto. Si el alumno quiere hacer a mano sus archivos HTML o CSS, es a gusto propio, esto no será evaluable.

2.3. Verificación de la práctica

Para verificar su práctica, el alumno deberá de actuar como un usuario final para su propio sistema y no deberá saltar ningún error inesperado, ni pantalla de error 5XX ni pantalla de *Internal System Error*, deberá de cumplir correctamente con los siguientes 3 flujos:

1. Inicio de sesión exitoso para usuario.
2. Inicio de sesión exitoso para administrador.
3. Inicio de sesión no exitoso.

Si dichos tres flujos funcionan correctamente, entonces la práctica se puede considerar como correcta

3. Preguntas

1. En el esquema de la base de datos proporcionada por los ayudantes, las contraseñas se guardan en texto plano, es decir, tal cual están almacenadas en la columna **password** corresponde a la contraseña de ese usuario, ¿por qué esto resulta inseguro desde el punto de vista de la ciberseguridad?

R= Por que si una persona ajena tiene acceso a la base de datos puede ver facilmente en la columna de password las contraseñas con el usuario asociado sin la necesidad de tratar de decifrarlo y muchos usuarios reutilizan sus contraseñas en diferentes servicios, por lo que un atacante podría utilizar esa información para acceder a otras cuentas del mismo usuario y comprometer aun más la información del usuario

2. Explica cómo es que una aplicación front-end debería conectarse con tu aplicación de Flask para que los templates no sean necesarios. Supón que tu aplicación no utiliza la función `render_template` ni `redirect`.

R= Por medio de solicitudes HTTP al backend usando herramientas como fetch donde el usuario ingresaria sus respectivos datos en el frontend y luego estos datos llegarían mediante una petición como POST al servidor Flask y con eso el servidor respondería si el login fue exitoso o no

3. ¿Qué son los CORS?

R= Es un mecanismo de seguridad que implementan los navegadores para controlar qué recursos pueden ser solicitados desde un dominio diferente al del servidor así se evitan ataques como el robo de información entre sitios web, pero también requiere configuración cuando se trabaja con arquitecturas separadas de frontend y backend.

4. En el contexto de aplicaciones back-end, ¿A qué nos referimos con *manejo de secretos*?

R= Se refiere a la forma en que una aplicación protege información sensible, como contraseñas, claves de API, tokens o credenciales de base de datos. En lugar de escribir estos datos directamente en el código se deben almacenar en variables de entorno o sistemas especializados de gestión de secretos lo que reduce el riesgo de exponer estos datos en repositorios publicos

5. Hablando del protocolo HTTP, describe un caso de uso en el que usar un método **GET** o un método **POST** no sea lo más ideal, es decir, un escenario en el que convenga usar un método que no sea ninguno de los mencionados.

R= Un caso donde no es ideal usar GET o POST es cuando se necesita actualizar parcialmente un recurso existente. En este escenario, es más adecuado utilizar el método PATCH. Otro caso sería la eliminación de un recurso, donde se debería utilizar el método DELETE, ya que ni GET ni POST están diseñados específicamente para esta operación.

4. Criterios de evaluación

Para esta práctica se evaluarán de manera equitativa tanto los flujos implementados por el alumno, como las preguntas, es decir los 3 flujos corresponderán al 50 % de la calificación y las 5 preguntas al porcentaje restante. El caracter de cada reactivo será de criterio correcto o incorrecto, es decir, 0 o 1 o cualquier cosa que se le asimile.

5. Lineamientos de entrega

Todo deberá de estar en una carpeta comprimida (cualquier formato), incluyendo el código fuente, con el nombre: **Practica4_<ApPat><ApMat>**, obviamente poniendo los respectivos apellidos paterno y materno.

La fecha de entrega para esta práctica es para el día 7 de Abril del 2026 antes de las 23:59:59.